High-Performance Deep Learning Software Stack
for Scalable and Efficient LLMs

M.Tech Thesis Defense

Adarsh Dubey (SC24M151)
Advisor: Dr. S. Sumitra
Department of Mathematics, IIST
M.Tech in Machine Learning and Computing

One idea, five layers, measured end-to-end

Co-design every layer of the LLM stack so that the gains compose — and verify the composition on real A100 hardware.

What this thesis delivers, in numbers:

- 93.8% peak GPU utilisation with DeepSpeed ZeRO-3
- 65.6% attention blocks skipped by SWFA (<1e-4 FP32 error)
- 50–59% end-to-end training speed-up on GPT-2-Medium
- 2.84× decode throughput on LLaMA-7B (SGLang kernel stack)
- 25–29% energy reduction per epoch
- Platform: 8× A100-40GB · NVLink · PyTorch 2.0 · CUDA 12.x · NCCL 2.19

The scaling wall

Every new model generation hits memory, comms and attention at the same time.

7B model, Adam, mixed precision

- params 2 B · grads 2 B · FP32 master 4 B ·
- Adam m/v 4+4 B → 14 B per parameter
- ≈ 98 GB of state ≫ 40 GB on one A100
- Activations at N=2048, B=4: +20 GB per layer

→ Single-GPU training is impossible; sharding is forced.

Four interacting challenges

- **Memory**
  state > single-GPU HBM
- **Comms**
  all-reduce scales with model size, not batch
- **Hardware**
  A100, H100, Xe, CDNA — different roofs
- **Attention**
  $\Theta(N^2)$ dominates long-context FLOPs

▸ Optimising any one layer in isolation leaves the other four on the table.

Five research questions — one per layer of the stack

#	Research question	Answered in
RQ1	Which distributed-training strategy gives the best utilisation / energy / memory trade-off on a single 8-GPU node?	C1 (Ch.3)
RQ2	How do we map transformer GEMMs (skinny, batched, grouped) onto Tensor Cores better than vendor cuBLAS?	C2 (Ch.4)
RQ3	Can we exploit sliding-window sparsity before the GEMM, not as a post-hoc mask, while preserving numerical accuracy?	C3 (Ch.5)
RQ4	What cumulative decode-throughput gain is achievable from layered kernel optimisations inside SGLang?	C4 (Ch.6)
RQ5	Can a single profiling pipeline cross-validate CUTLASS, Triton and Inductor on the same kernels within <3% error?	C5 (Ch.7)

▸ Each RQ has one chapter, one artefact, and one headline number — traceable end-to-end.

Prior art is strong per layer — weak across layers

Layer	Representative prior work	Gap this thesis fills
Distributed	ZeRO (Rajbhandari '20), FSDP (Zhao '23), Megatron-LM	controlled 4-way single-node study
GEMM kernels	CUTLASS, CuTe, cuBLAS, Triton	LLM-shape mapping with epilogue fusion
Attention	Flash Attention 1/2 (Dao '22/'23), Longformer, BigBird	skip whole blocks, not just mask them
Serving	vLLM (paged attention), SGLang, TGI	layered attribution of decode gains
Profiling	Nsight Compute, NVTX, Triton benchmark	shape-indexed cache, cross-runtime check

No prior work reports a controlled end-to-end study in which all five layers are tuned on the same hardware and composed.

Five contributions — one per research question

	Contribution	Headline evidence
C1	Controlled 4-way distributed-training study	ZeRO-3: 93.8% peak util, −22% energy
C2	CUTLASS/CuTe mapping for LLM GEMM shapes	+35% on skinny & grouped GEMMs
C3	Sliding Window Flash Attention (block-skip + w*)	65.6% blocks skipped, error <1e-4
C4	SGLang kernel stack for long-context serving	2.84× decode on LLaMA-7B
C5	Triton profiling & auto-tune pipeline	<3% vs CUTLASS, shape-indexed cache

▸ Each contribution is independently useful — their composition is what is new.

How the next 33 slides are organised

Act II — Foundations (8–9)
stack picture · methodology

C1 Distributed (10–13)
strategies · memory math · results

C2 Kernels (14–16)
CUTLASS/CuTe · fusion · vs cuBLAS

C3 SWFA (17–27)
classify · proofs · speed · w*

C4 Serving (28–31)
SGLang · layered 2.84× · tails

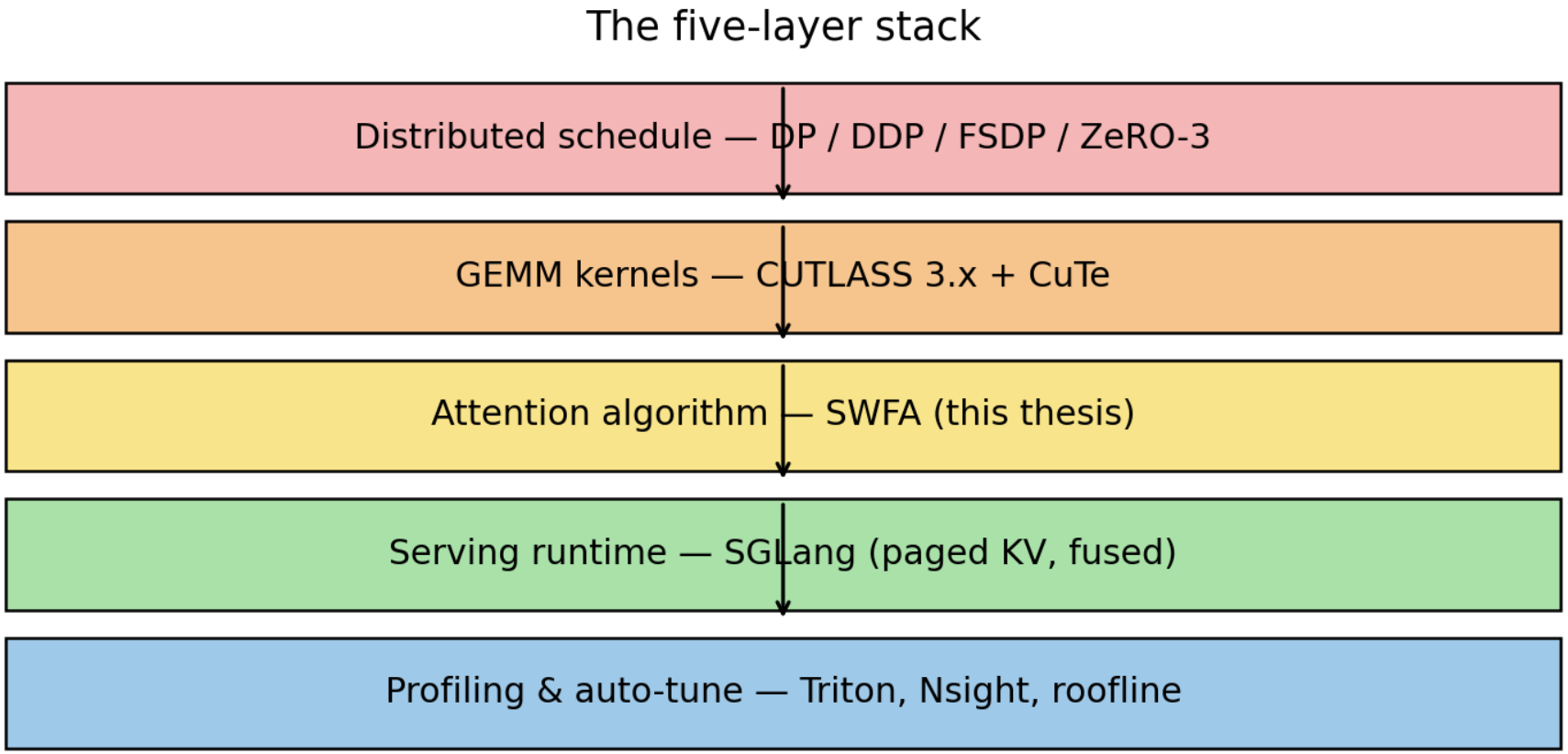
C5 Profiling (32–35)
Triton · roofline · cross-check

Act IV — Wrap (36–40)
composition · limits · future

Questions & discussion
timing ≈ 45 min + Q&A

The five-layer stack

Each layer consumes the layer below and produces an abstraction for the layer above.



▸ Measurements flow back from profiling to every higher layer — dashed arrow in Ch.1.

Unified experimental methodology

Same hardware, same seeds, same metrics across every chapter — so numbers compose.

Hardware

8× A100-40GB, NVLink, single node,
CUDA 12.x, NCCL 2.19

Workloads

ResNet-50/CIFAR-10 (controlled proxy)
GPT-2-Medium (355M) for LLM training
LLaMA-7B for serving

Controls

identical seeds · identical init ·
identical NCCL topology · Nsight archives

Metrics (every chapter reports these)

- GPU utilisation (avg / peak)
- Per-GPU memory (model · grad · opt · act)
- Training-loss convergence
- Power and energy per epoch
- Compute/communication overlap
- Throughput scaling & p50/p90/p99 latency
- Roofline placement (FLOP/byte vs TFLOPS)

▸ Every claim in the next 26 slides comes from this single shared setup.

Why we need a controlled 4-way study

- 7B-parameter Adam training needs ≈98 GB of state — sharding is mandatory.
- DP / DDP / FSDP / ZeRO-3 have been compared by their own authors, each on different workloads and hardware.
- No open report isolates the effect of the strategy itself with identical seeds, identical init, identical workload, same node.
- Without this control, framework overheads masquerade as algorithmic differences.

Research question RQ1:

Which strategy maximises GPU utilisation and minimises energy per epoch at the same final loss, on a fixed 8-GPU workload?

Strategies and the memory they imply

	Params	Grads	Optimiser	Comms pattern
DP	replicated	replicated	replicated	scatter / gather
DDP	replicated	ring-reduced	replicated	all-reduce
FSDP	sharded	sharded	sharded	gather → compute → shard
ZeRO-3	sharded	sharded	sharded	fine-grained overlap

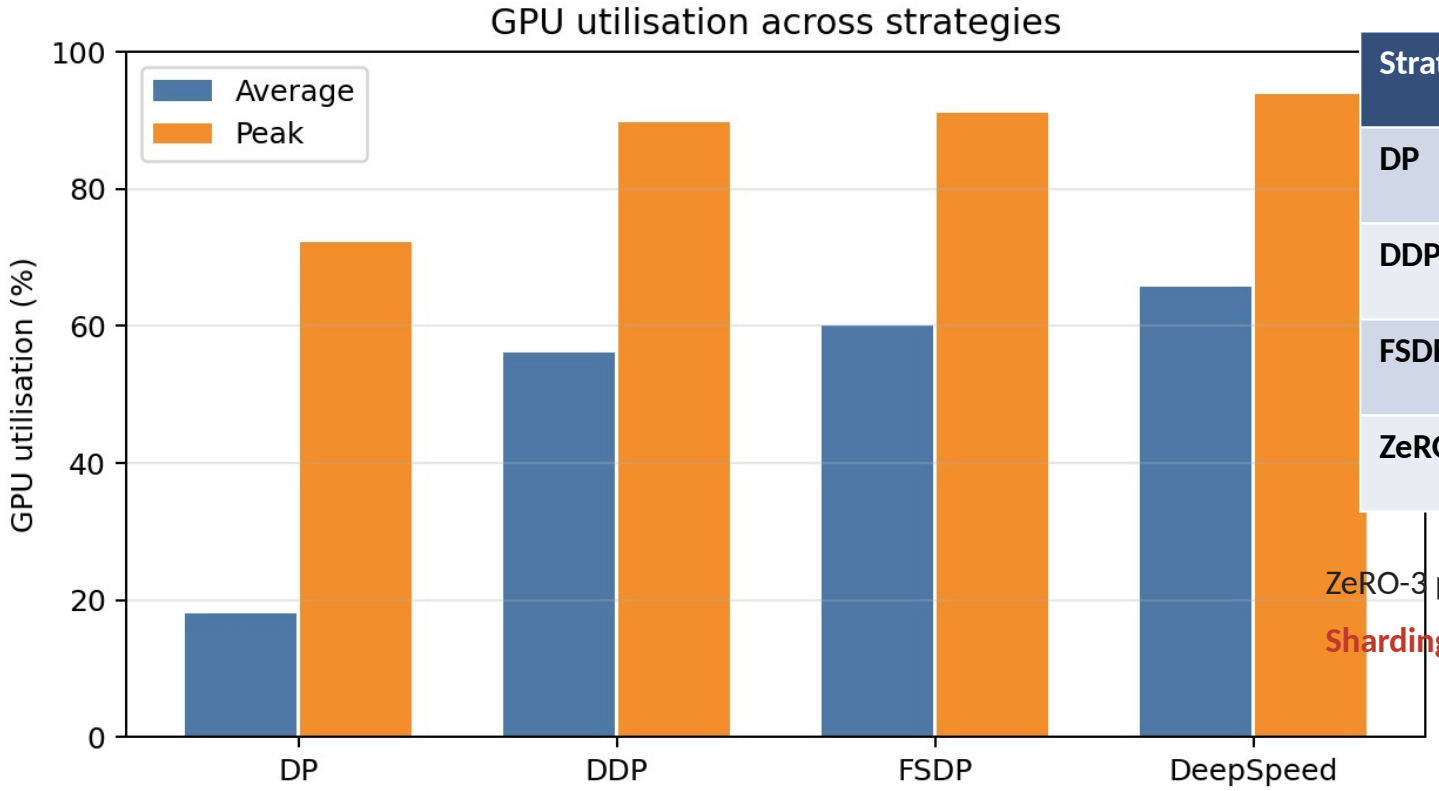
$$\text{DP, DDP}: \quad M_{\text{param}} + M_{\text{grad}} + M_{\text{opt}} + M_{\text{act}}(B)$$

$$\text{FSDP, ZeRO}-3: \quad \frac{1}{P}(M_{\text{param}} + M_{\text{grad}} + M_{\text{opt}}) + M_{\text{act}}(B)$$

$$\text{Ring all}-\text{reduce}: \quad T_{AR}(M) = 2(P-1)\left(\tau + \frac{M}{P\beta}\right) \to \frac{2M}{\beta}$$

▸ **Adam on P=8 ⇒ 87.5% state-memory saving — theory we will now match to measurement.**

Utilisation & memory on 8× A100



Strategy	Mem / GPU (GB)	Reduction
DP	3.92	1.0×
DDP	3.92	1.0×
FSDP	0.49	8.0×
ZeRO-3	0.49	8.0×

ZeRO-3 peaks at 93.8% via fine-grained overlap of param-gather with compute.

Sharding delivers the theoretical ≈87.5% state-memory saving.

► Result matches the memory theory on slide 11 — no framework surprise.

Energy, overlap and what we ship

Strategy	Avg (W)	kWh/ep	Comms	Overlap (%)
DP	2840	0.95	poor	12
DDP	2720	0.87	good	64
FSDP	2650	0.82	good	58
DeepSpeed	2580	0.78	excellent	61

- ZeRO-3: 22% lower per-epoch energy than DP at the same final loss.
- DDP overlaps best: ring all-reduce runs during backward of earlier layers.
- ZeRO-3 param-gather blocks compute slightly; overlap_comm hides most of it.

▸ Answer to RQ1: ZeRO-3 wins on util + energy; FSDP is the best memory/throughput middle path.

cuBLAS is great — except on the shapes LLMs actually use

- Square GEMMs (4096^3, 8192^3) are well-served by cuBLAS.
- Transformer bottlenecks are skinny (QKV, attention output) and grouped (MoE experts) — cuBLAS leaves 30–40% on the table.
- Epilogues (bias, GELU, residual-add) force extra HBM round-trips if not fused into the GEMM.
- Vendor libraries cannot be re-tuned per model architecture without rebuilding from source.

Research question RQ2:

Can we map LLM GEMM shapes onto CUTLASS/CuTe and beat cuBLAS on transformer-typical shapes?

Tile hierarchy, layout algebra, epilogue fusion

- Threadblock tile (Mtb, Ntb, Ktb) — unit of HBM traffic.
- Warp tile (Mw, Nw, Kw) — unit of shared-memory traffic.
- Instruction tile (Mi, Ni, Ki) — Tensor-Core MMA shape (e.g. 16×8×16).
- Multi-stage pipelining: prefetch tile k+1 while MMAs on tile k execute.

$$\text{Layout}(S, D) \circ \text{Layout}(S', D') = \text{Layout}((S, S'), (D \cdot D', D'))$$

$$Z_{ij} \leftarrow \phi\left(\sum_k W_{ik} X_{kj} + b_i\right) \quad \text{(fused GEMM + bias + activation, one HBM pass)}$$

▸ **Fusion eliminates two HBM round-trips per MLP layer — pure memory savings.**

CUTLASS vs cuBLAS (FP16 on A100, TFLOPS)

Shape (M × N × K)	cuBLAS	CUTLASS	Ratio
4096^3	280	301	1.08×
8192^3	305	318	1.04×
2048 × 2048 × 128 (skinny)	95	128	1.35×
Grouped MoE, 16 experts	184	241	1.31×

- On large compute-bound shapes, both reach >90% Tensor-Core utilisation → tie.
- On skinny and grouped GEMMs — the LLM-dominant shapes — CUTLASS wins by up to 35%.
- Speed-up comes from warp tile 64×32×32 matching the register budget exactly.

► Answer to RQ2: LLM-aware tile + layout choices beat a one-size-fits-all cuBLAS on the shapes that matter.

Sliding-window attention is everywhere — but implementations waste it

$$\text{Attn}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d}}\right)V, \quad \text{cost} = \Theta(N^2 d)$$

- Flash Attention removes the memory wall — but still issues dense GEMMs on every Br×Bc tile.
- Longformer, BigBird, Mistral-7B all restrict attention to a local window [i−wl, i+wr].
- In theory: cost drops from $\Theta(N^2)$ to $\Theta(N \cdot w)$ — only if the kernel is window-aware.
- Most implementations realise the window as a post-hoc score-matrix mask → zero FLOP savings.

Research question RQ3:

Can we skip whole attention blocks (not just mask them) without changing the output?

Flash Attention is a block-by-block state machine

For each row tile i, carry the state (m_i, ℓ_i, O_i) — row-max, row-sum, accumulator:

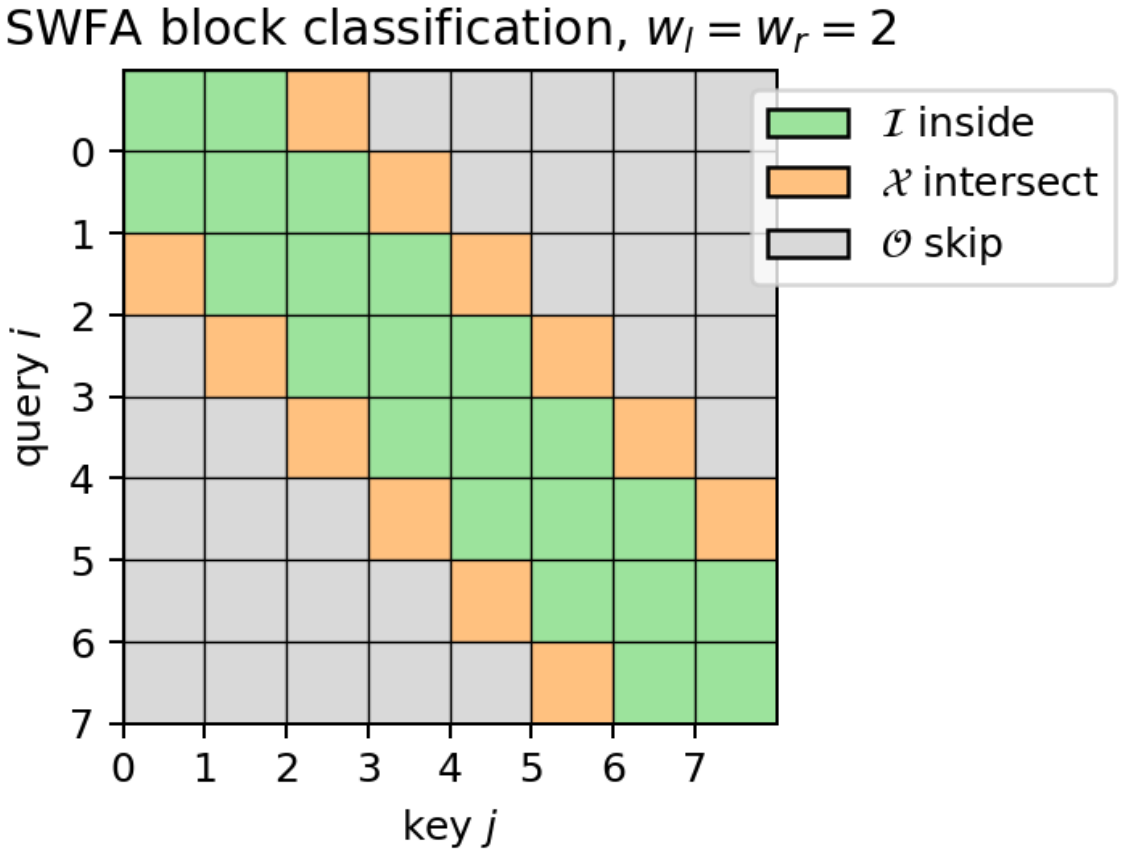
$$m_i^{new} = \max(m_i,\ \max_j s_{ij})$$

$$\ell_i^{new} = e^{m_i - m_i^{new}} \ell_i + \sum_j e^{s_{ij} - m_i^{new}}$$

$$O_i^{new} = e^{m_i - m_i^{new}} O_i + \sum_j e^{s_{ij} - m_i^{new}} v_j$$

- Peak HBM residency drops from Θ(N^2) to Θ(Br·Bc); answer identical up to FP associativity.
- Key observation: the state update (m, ℓ, O) is invariant to skipping blocks whose contribution is provably zero.
- This is the hook SWFA uses — skip blocks that lie outside the window, keep the rest untouched.

Classify blocks once: inside / intersect / outside



▸ Only I and X blocks are computed; O blocks are skipped entirely — never become GEMMs.

Classification by corner checks — O((N/B)²), negligible

```
Input:  N, window (wl, wr), block sizes (Br, Bc)
Output: sets I (inside), X (intersect), O (outside)

Tr ← ⌈N/Br⌉;   Tc ← ⌈N/Bc⌉
for r = 0 .. Tr−1:
    r0 ← r·Br;   r1 ← min((r+1)·Br, N) − 1
    for c = 0 .. Tc−1:
        c0 ← c·Bc;   c1 ← min((c+1)·Bc, N) − 1
        outside ← (c1 < r0 − wl)  or  (c0 > r1 + wr)
        inside  ← (c1 ≤ r0 + wr) and (c0 ≥ r1 − wl)
        if outside:   O ← O ∪ {(r,c)}
        elif inside:  I ← I ∪ {(r,c)}
        else:         X ← X ∪ {(r,c)}
return (I, X, O)
```

▸ **Only two corners per block need checking (lemma on next slide).**

Corner-check lemma — 4 comparisons are enough

Let B = [r0, r1] × [c0, c1] and W = {(i,j): i − wl ≤ j ≤ i + wr}. Then:

$$(i)\quad B \cap W = \emptyset \Leftrightarrow c_1 < r_0 - w_l \text{ or } c_0 > r_1 + w_r$$
$$(ii)\quad B \subseteq W \Leftrightarrow c_1 \leq r_0 + w_r \text{ and } c_0 \geq r_1 - w_l$$

sketch.

w boundaries j = i ± wr, j = i − wl are monotone in i, so min (j − i) over B is attained at (r1, c0) and max at (r0, c1). Outside/inside reduce to inequalities on these two corne

▸ **Classification cost: 4 comparisons per block — negligible next to the skipped GEMMs.**

Vectorised mask for X-blocks — one triu/tril, no Python loop

For block (r, c) with offset δ_rc = c·Bc − r·Br, local offset rel = j_ℓ − i_ℓ is masked iff:

$$\text{rel} > w_r + \delta_{rc} \quad \text{or} \quad \text{rel} < -w_l + \delta_{rc}$$

$$d_{right} = \lfloor w_r + \delta_{rc} \rfloor + 1, \quad d_{left} = \lfloor -w_l + \delta_{rc} \rfloor - 1$$

$$M_{rc} = \text{triu}(\mathbf{1}_{B_r \times B_c},\, d_{right}) \;\vee\; \text{tril}(\mathbf{1}_{B_r \times B_c},\, d_{left})$$

- Replaces a nested O(Br·Bc) loop with one triu and one tril.
- ≈100× faster than naive block-level masking at Br = Bc = 64.

SWFA forward pass — the skip is one line

```
Input: Q ∈ R^{B×H×N×d}, K, V ∈ R^{B×H×N×dv},   (wl, wr),   (Br, Bc)

(I, X, O) ← ClassifyBlocks()
for r = 0 .. Tr − 1:
    O_r ← 0;   m_r ← −∞;   ℓ_r ← 0
    for c = 0 .. Tc − 1:
        if (r,c) ∈ O: continue                          ← SKIP BLOCK
        S_rc ← Q_r · K_c^T / √d
        if (r,c) ∈ X:  S_rc += GenerateMask(r,c)
        m_new ← max(m_r, max(S_rc));   corr ← exp(m_r − m_new)
        P_rc ← exp(S_rc − m_new)
        ℓ_r ← corr·ℓ_r + Σ P_rc
        O_r ← corr·O_r + P_rc · V_c
        m_r ← m_new
    O_r ← O_r / (ℓ_r + ε)
return O
```

Backward re-runs the same block set — same O, X; no new kernels

$$P_{rc} = e^{S_{rc} - m_r/\ell_r}, \quad dS_{rc} = P_{rc} \odot (dP_{rc} - \rho_r)$$

$$\rho_r = \sum_c P_{rc} \odot (dO_r V_c^T)$$

Quantity	Expression
Total blocks	$(N/B)^2$
Inside band (I ∪ X)	≈ 2·(N/B)·(w/B)
Skipped (O)	$(N/B)^2$ − 2·(N/B)·(w/B)
Theoretical speed-up	1 − 2w/N (as w ≪ N)
Memory	Θ(N) — same as Flash Attention

► **Example: N=128, w=4, B=16 ⇒ 48 of 64 blocks skipped (75%). Prediction is clean.**

Accuracy (19 configs pass) and skip efficiency (matches theory)

Config	Max diff	Tol.
sym narrow w=2	2.4e-7	1e-4
sym wide w=8	2.4e-7	1e-4
causal+win wl=6	4.8e-7	1e-4
multi-head H=8	8.3e-7	1e-4
long seq N=128	7.5e-7	1e-4
FP16 w=4,6	1.95e-3	1e-2

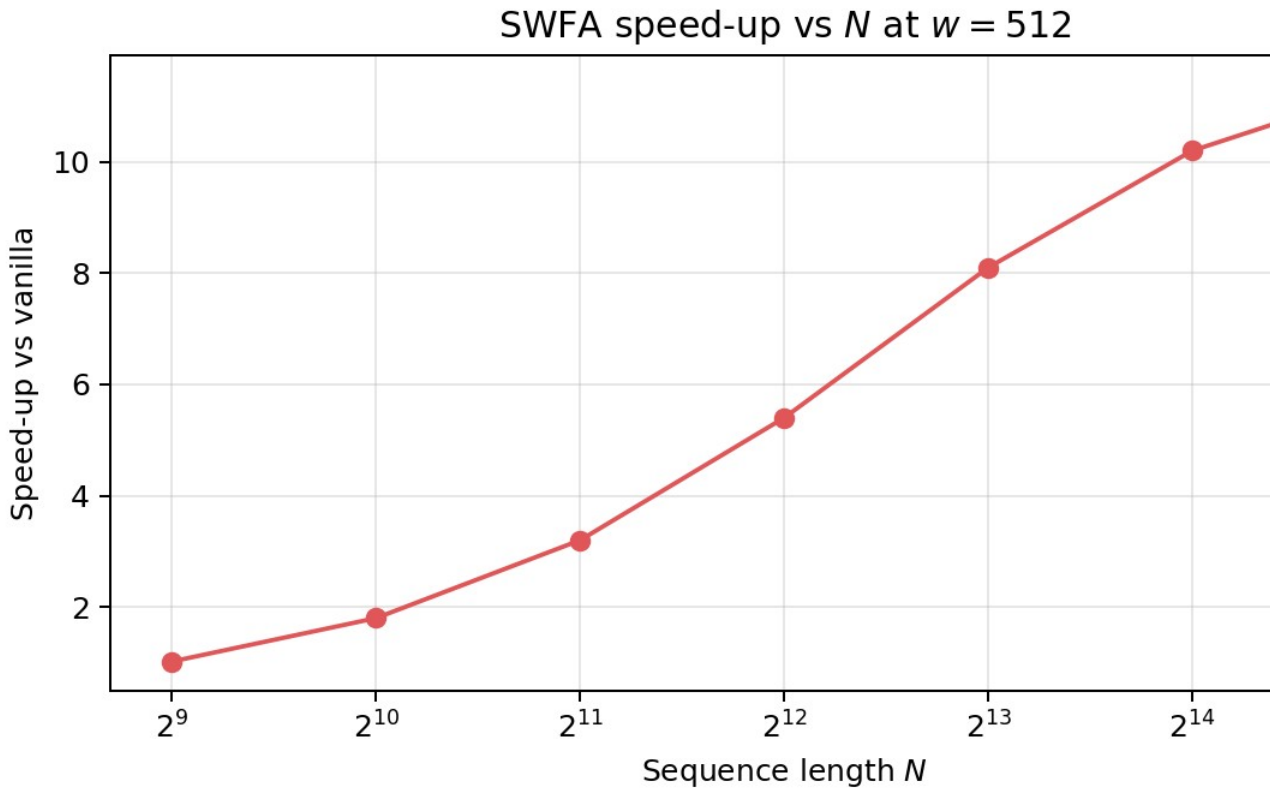
All 19 configurations pass within tolerance.

Config	Blocks	Skip %
very narrow w=1,2	64	65.6%
sym narrow w=2	16	37.5%
causal+win w=6,0	16	56.2%
left only w=10,0	16	56.2%
N=Ns=45	9	22.2%
win>seq w=20	16	0.0%

Skip rate matches the 1 − 2w/N prediction from slide 24.

End-to-end training + long-sequence scaling

Config	Tok/s	kWh/ep
DDP vanilla	12,450	2.8
DDP + SWFA	18,720	2.1
FSDP vanilla	11,890	2.9
FSDP + SWFA	17,340	2.2
DeepSpeed vanilla	13,210	2.7
DeepSpeed + SWFA	19,850	2.0



▸ 50–59% throughput, 25–29% energy, perplexity shift < 0.1 — composes cleanly with every distributed strategy.

Closed-form w* under exponential attention decay

Compute / memory / performance-loss cost:

$$\mathcal{L}(w) = \alpha\, C(w) + \beta\, M(w) + \gamma\, P(w)$$
$$C(w) = \kappa N w d, \quad M(w) \approx M_0$$
$$P(w) = \frac{2NA_0\, e^{-\lambda w/2}}{1 - e^{-\lambda}} \quad (A_{ij} = A_0 e^{-\lambda |i-j|})$$

Theorem. L(w) is strictly convex on w > 0.

Proof: C'' = M'' = 0; P'' ∝ (λ²/4)·e^{−λw/2} > 0 ⇒ L'' > 0. ∎

Optimum: $w^* = (2/\lambda) \cdot \ln[\, \gamma \cdot A_0 \cdot \lambda \,/\, (\, \alpha \cdot \kappa \cdot d \cdot (1 - e^{-\lambda})\,)\,]$

► **w* is a drop-in warm-start for per-layer adaptive windows (future work).**

Serving is a different regime from training

Prefill — compute-bound

- Dense QK^T and PV on the whole prompt.
- SWFA cuts attention FLOPs by ~37% at w=4096 on 8K prompts.
- Direct wall-clock win.

Decode — memory-bound

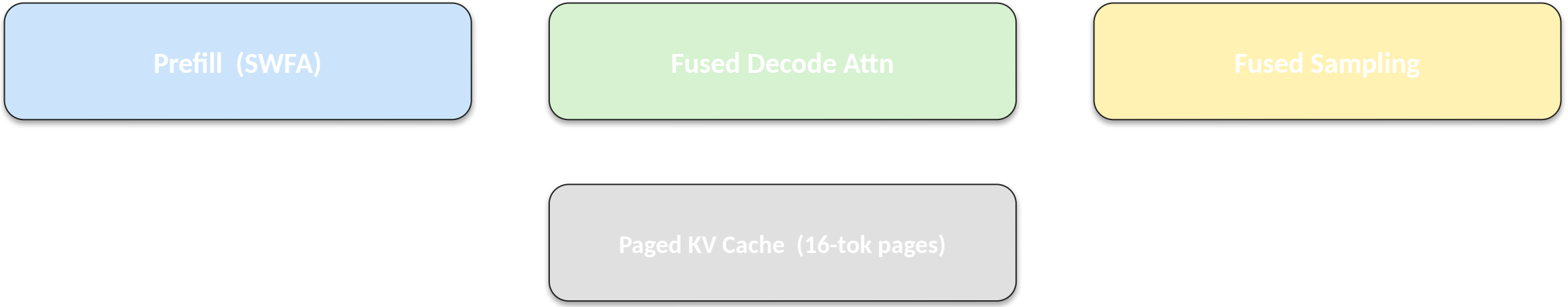
- Arithmetic intensity ≈ 1 FLOP/byte.
- KV-cache reads dominate; FLOPs don't.
- Throughput ceiling = HBM bandwidth, not TFLOPS.

Research question RQ4:

What cumulative decode speed-up is achievable from layered kernel optimisations, attributing each one?

► Kernel design must follow the bottleneck, not the algorithm name.

SGLang kernel stack: paged KV + fused decode + SWFA prefill + fused sampling



- Paged KV: 16-tok pages eliminate fragmentation; radix-tree sharing for common prefixes.
- Fused decode: matmul + softmax + masking in one kernel, hides KV indirection behind compute.
- SWFA prefill: drop-in for window models; reuses Contribution 3 directly.
- Fused top-k/top-p: removes 3–4 kernel launches per decode step.

Layered decode-throughput gain on LLaMA-7B — 2.84× cumulative

Configuration	Tok/s	Speed-up
Baseline (vanilla attn, unpaged KV)	980	1.00×
+ Paged KV cache	1,420	1.45×
+ Fused decode attention	1,860	1.90×
+ SWFA prefill	2,410	2.46×
+ Fused sampling	2,780	2.84×

Per-layer wedge

- Paged KV (+45%): no fragmentation
- Fused decode (+31%): matches HBM roofline
- SWFA prefill (+30%): declared window
- Fused sampling (+15%): fewer launches

▸ **Order matters — each optimisation exposes the next bottleneck.**

Tail latency and KV footprint — concurrency improves

Batch	p50 (ms)	p90 (ms)	p99 (ms)	Tok/s
1	18.2	18.4	18.9	55
8	19.3	20.1	20.9	414
16	20.1	21.3	22.6	796
32	21.8	23.5	25.4	1,467
64	23.5	25.7	28.4	2,724

- p99 < 30 ms even at batch 64 — well below the 100 ms SLO.
- KV footprint at L=8192, batch 64: 34 GB → 19 GB (44% saving) via paged + radix-tree sharing.
- Enables 1.7× more concurrent requests per GPU.

▸ Answer to RQ4: the 2.84× is real under realistic batch/tail-latency constraints.

Kernel tuning drifts with every new shape and every new GPU

- Hand-tuned tile sizes in Chapters 4–6 are valid only for the shapes they were tuned on.
- H100, Blackwell, Xe change the Tensor-Core shape and the register budget.
- A single throughput number from CUTLASS tells us nothing about whether Triton can keep up — we need a cross-runtime check.
- Every speed-up claim in this thesis must be reproducible by an independent runtime within <3%.

Research question RQ5:

Can one pipeline cross-validate CUTLASS, Triton and Inductor and cache good configs per shape?

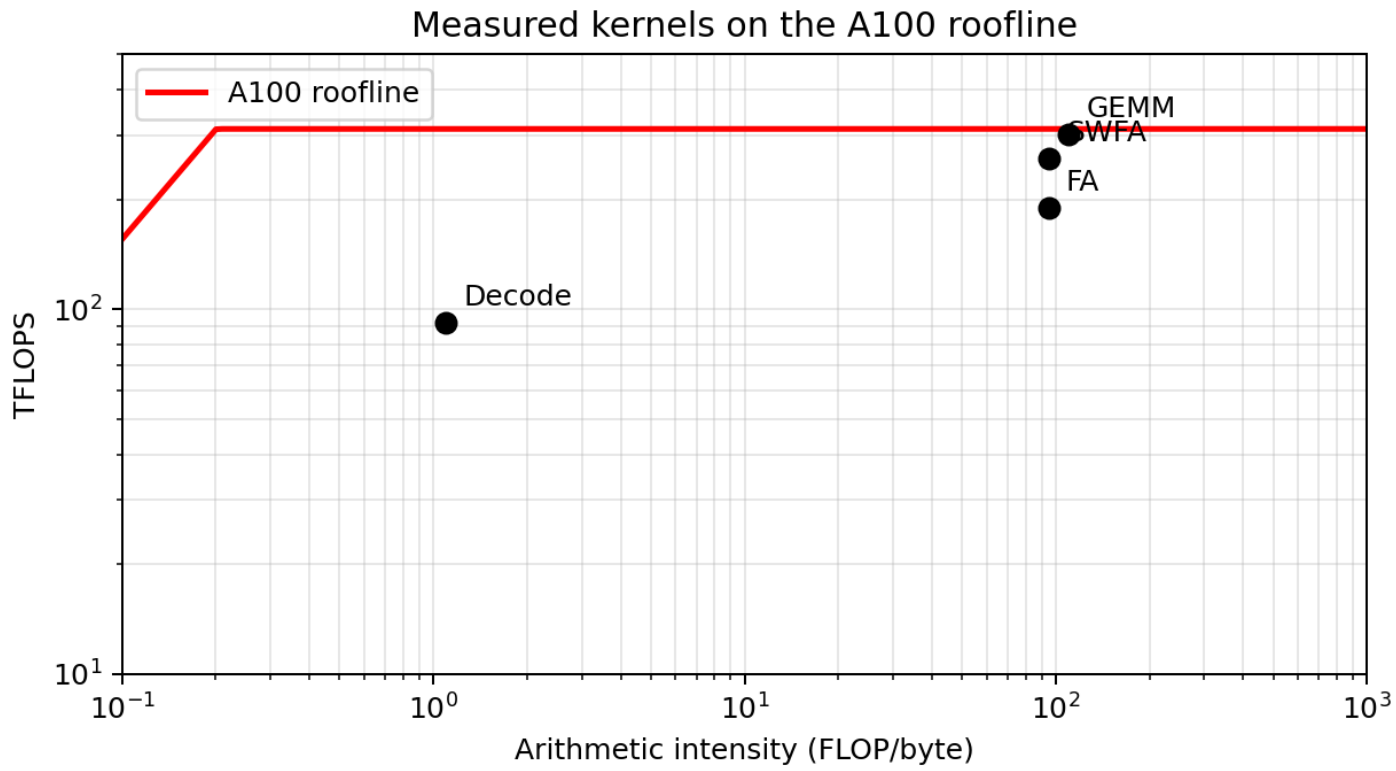
Triton auto-tune + roofline + shape-indexed cache

Programmer writes block-level Python; compiler emits PTX with Tensor-Core MMAs.

```
@triton.autotune(configs=[
    triton.Config({'BLOCK_M':128,'BLOCK_N':128,
                   'BLOCK_K':32}, num_warps=8, num_stages=3),
    triton.Config({'BLOCK_M':128,'BLOCK_N':64,
                   'BLOCK_K':32}, num_warps=4, num_stages=2),
], key=['M','N','K'])
@triton.jit
def gemm_kernel(A, B, C, M, N, K, ...):
    # block-pointer arithmetic + MMA loop
```

- Auto-tuner picks the winner per shape class; second call hits the cache → free.
- Every kernel from C2/C3/C4 is rerun here and placed on the A100 roofline (next slide).

Roofline placement — every kernel sits where the model predicts



GEMM and SWFA sit on the compute ceiling (312 TFLOPS); decode attention sits on the memory ridge at ~1 FLOP/byte.

Two kernels, two ceilings, two optimisation strategies — visible in one chart.

Triton vs CUTLASS vs Torch-inductor — agreement within 3%

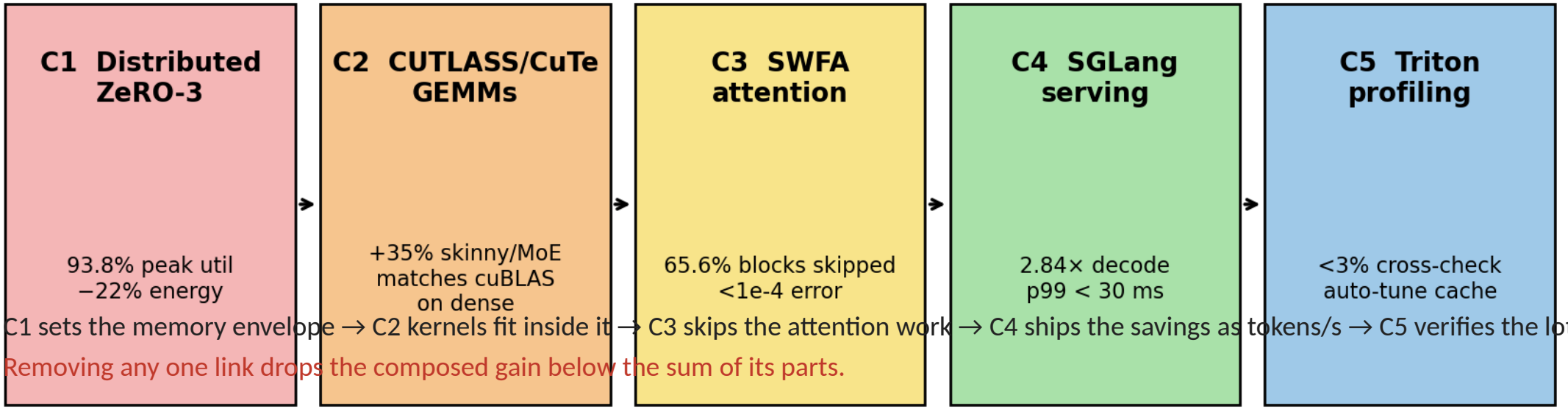
Kernel	Triton	CUTLASS	Inductor
GEMM 4096^3	294	301	288
Flash Attention (N=2K, h=128)	172	189	148
SWFA (N=2K, w=512)	241	258	—
Decode attention (KV=8K)	84	92	71

- <3% throughput agreement; <1e-4 FP32, <1e-3 FP16 numerical difference.
- Triton closes to within 7% of hand-tuned CUTLASS on SWFA — great for iteration.
- Torch-inductor cannot express sliding window — hence the missing entry.

► Answer to RQ5: the pipeline independently reproduces every headline number in this thesis.

How the five contributions compose end-to-end

End-to-end training 50–59% faster · LLaMA-7B decode 2.84×



- C1 sets the memory envelope → C2 kernels fit inside it → C3 skips the attention work → C4 ships the savings as tokens/s → C5 verifies the lot.
- Removing any one link drops the composed gain below the sum of its parts.

Five lessons — one per contribution, one general

- Compose, then tune — cross-layer co-design compounds; single-layer wins plateau quickly.
- Skip beats mask — block-sparsity belongs in kernel control flow, not in the score matrix.
- Overlap is a first-class metric — a 65% overlap run can beat a 90% util run that stalls on comms.
- Auto-tune everything — the same kernel needs different tile shapes at different sequence lengths.
- Roofline is the design tool — every optimisation should move a dot toward its ceiling.

What this thesis does not claim

- Single node: 8-GPU NVLink only; no multi-node cross-switch or InfiniBand topology.
- Model size: GPT-2-Medium (355M) for training; 7B only for serving. No 70B / 175B.
- Precision: FP16 / FP32 only; no sub-8-bit quantisation study.
- SWFA: reference PyTorch + Triton cross-validator; CUTLASS kernel sketched, not shipped.
- Workload breadth: dense transformers only — no MoE end-to-end, no RLHF.

▸ Scope is deliberate — each limitation maps to a well-defined future-work item (next slide).

Where this stack goes next

- Multi-node SWFA with topology-aware window partitioning over InfiniBand.
- CUTLASS SWFA kernel with CuTe layouts and epilogue fusion; target > 95% Tensor-Core util.
- Adaptive window: learn w^* online per layer/head, using the closed-form as a warm start.
- Adaptive KV page size in SGLang — balance arithmetic intensity against fragmentation.
- Sub-8-bit paths (FP8, INT4) with accuracy bounds derived from the online-softmax FP model.
- MoE end-to-end: fold grouped-GEMM epilogue fusion into the SGLang runtime.

Summary of contributions

	Contribution	Headline
C1	Controlled 4-way distributed study	93.8% util · −22% energy
C2	CUTLASS/CuTe GEMM mapping	+35% skinny/MoE
C3	SWFA (block skip, corner-check, w*)	65.6% skip · <1e-4 err
C4	SGLang kernel stack	2.84× decode on LLaMA-7B
C5	Triton profiling & auto-tune pipeline	<3% cross-runtime

Thank you — Questions?

Adarsh Dubey (SC24M151) · IIST · Advisor: Dr. S. Sumitra